

Research Notebook — *Dean Kulik's Unified Theoretical Framework*

This notebook is a **research companion** to the paper, not a replacement for it.

It extracts the parts of the framework that admit direct execution, measurement, or visualization:

- the ontological inversion scaffold,
- the 9-primitive action basis and its 81-pair operator tensor,
- the dual-channel SHA-256 die,
- the A-Mark9 ground witness and carry geometry,
- the topological waist,
- the Sarrus isomorphism crosswalk,
- BBP random-access word extraction,
- the Mark 1 attractor ($H = \pi/9$).

The notebook keeps the implementation **portable**:

- install cell first,
- no hard-coded directories,
- optional exports only,
- no solver stack.

```
In [1]: %pip install -q mpmath numpy pandas plotly nbformat
```

Note: you may need to restart the kernel to use updated packages.

```
In [2]: from __future__ import annotations
```

```
from pathlib import Path
import os
import math
import time
import random
```

```

import statistics
from decimal import Decimal, getcontext
from typing import Dict, List, Tuple

import mpmath as mp
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

# -----
# Portable notebook configuration
# -----
EXPORT_ARTIFACTS = False
OUTPUT_DIR = Path.cwd() / "unified_framework_outputs"
RANDOM_SEED = 42

random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
mp.mp.dps = 120

if EXPORT_ARTIFACTS:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 160)

MASK32 = 0xFFFFFFFF
MOD = 1 << 32
PHI_CONJ = (math.sqrt(5.0) - 1.0) / 2.0
MARK1_H = math.pi / 9.0

H0 = [
    0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
    0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19,
]

K64 = [
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,

```

```

0x27b70a85,0x2e1b2138,0x4d2c6dfc,0x53380d13,0x650a7354,0x766a0abb,0x81c2c92e,0x92722c85,
0xa2bfe8a1,0xa81a664b,0xc24b8b70,0xc76c51a3,0xd192e819,0xd6990624,0xf40e3585,0x106aa070,
0x19a4c116,0x1e376c08,0x2748774c,0x34b0bcb5,0x391c0cb3,0x4ed8aa4a,0x5b9cca4f,0x682e6ff3,
0x748f82ee,0x78a5636f,0x84c87814,0x8cc70208,0x90beffff,0xa4506ceb,0xbef9a3f7,0xc67178f2
]

PRIMITIVES = [
    "PROJECT", "REFLECT", "FOLD", "LEAK", "GATE",
    "BRANCH", "PIN", "SYNC", "VERIFY"
]

def rotr(x, n):
    return ((x >> n) | ((x << (32 - n)) & MASK32)) & MASK32

def Sigma0(x):
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sigma1(x):
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sigma0(x):
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def sigma1(x):
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def ch(e, f, g):
    return ((e & f) ^ ((~e) & g)) & MASK32

def maj(a, b, c):
    return (a & b) ^ (a & c) ^ (b & c)

def add32(*xs):
    return sum(xs) & MASK32

def hw(x):
    return int(x).bit_count()

def pad_single_block(message: bytes) -> bytes:
    if len(message) > 55:
        raise ValueError("This notebook uses single-block messages only (<= 55 bytes).")
    bit_len = len(message) * 8

```

```

padded = message + b"\x80"
while len(padded) % 64 != 56:
    padded += b"\x00"
padded += bit_len.to_bytes(8, "big")
return padded

def words_from_message(message: bytes):
    padded = pad_single_block(message)
    return [int.from_bytes(padded[i:i+4], "big") for i in range(0, 64, 4)]

def msg_schedule(words16):
    W = list(words16) + [0] * 48
    for r in range(16, 64):
        W[r] = add32(sigma1(W[r-2]), W[r-7], sigma0(W[r-15]), W[r-16])
    return W

def exact_carry_word(x, y):
    carry_word = 0
    c = 0
    for i in range(32):
        xi = (x >> i) & 1
        yi = (y >> i) & 1
        c = (xi & yi) | (xi & c) | (yi & c)
        if c:
            carry_word |= (1 << i)
    return carry_word

def carry_out_count(*xs):
    return (sum(xs) >> 32)

def compress_trace(words16, rounds=64):
    W = msg_schedule(words16)
    a, b, c, d, e, f, g, h = H0
    rows = []
    for r in range(rounds):
        t1 = add32(h, Sigma1(e), ch(e, f, g), K64[r], W[r])
        t2 = add32(Sigma0(a), maj(a, b, c))

        carry_t1_word = 0
        acc = 0
        for item in [h, Sigma1(e), ch(e, f, g), K64[r], W[r]]:
            carry_t1_word |= exact_carry_word(acc, item)

```

```

        acc = add32(acc, item)

    carry_t2_word = exact_carry_word(Sigma0(a), maj(a, b, c))
    a_next = add32(t1, t2)
    e_next = add32(d, t1)

    rows.append({
        "round": r,
        "a": a, "b": b, "c": c, "d": d, "e": e, "f": f, "g": g, "h": h,
        "W": W[r], "T1": t1, "T2": t2,
        "carry_T1_count": carry_out_count(h, Sigma1(e), ch(e, f, g), K64[r], W[r]),
        "carry_T2_count": carry_out_count(Sigma0(a), maj(a, b, c)),
        "carry_T1_word": carry_t1_word,
        "carry_T2_word": carry_t2_word,
        "carry_T1_hw": hw(carry_t1_word),
        "carry_T2_hw": hw(carry_t2_word),
        "a_next": a_next,
        "e_next": e_next,
        "sziklai_residual": ((a_next - e_next) - (t2 - d)) & MASK32,
    })
    a, b, c, d, e, f, g, h = a_next, a, b, c, e_next, e, f, g
    return pd.DataFrame(rows)

def random_one_block_message():
    n = random.randint(1, 55)
    return os.urandom(n)

def primes(n):
    out = []
    x = 2
    while len(out) < n:
        good = True
        for p in out:
            if p * p > x:
                break
            if x % p == 0:
                good = False
                break
        if good:
            out.append(x)
        x += 1
    return out

```

```

def cube_root_fraction_word(p):
    getcontext().prec = 100
    x = Decimal(p) ** (Decimal(1) / Decimal(3))
    frac = x - int(x)
    return int(frac * (1 << 32))

def bbp_hex_digit(d: int) -> int:
    if d < 0:
        raise ValueError("d must be non-negative")
    s = mp.mpf("0")
    for j, coeff in [(1, 4), (4, -2), (5, -1), (6, -1)]:
        sj = mp.mpf("0")
        for k in range(d + 1):
            denom = 8 * k + j
            sj += mp.mpf(pow(16, d - k, denom)) / denom
        k = d + 1
        term = mp.power(16, d - k) / (8 * k + j)
        while abs(term) > mp.mpf("1e-80"):
            sj += term
            k += 1
            term = mp.power(16, d - k) / (8 * k + j)
        s += coeff * sj
    frac = s - mp.floor(s)
    return int(mp.floor(16 * frac))

def bbp_word32(d: int) -> int:
    word = 0
    for i in range(8):
        word = (word << 4) | bbp_hex_digit(d + i)
    return word

def word_support_history(initial_word_index=0, rounds=8):
    dep = {lane: set() for lane in "abcdefgh"}
    Wdep = {i: ({i} if i == initial_word_index else set()) for i in range(64)}
    for r in range(16, 64):
        Wdep[r] = Wdep[r-2] | Wdep[r-7] | Wdep[r-15] | Wdep[r-16]

    rows = []
    for r in range(rounds):
        t1dep = dep["h"] | dep["e"] | dep["f"] | dep["g"] | Wdep[r]
        t2dep = dep["a"] | dep["b"] | dep["c"]

```

```

        dep_next = {
            "a": t1dep | t2dep,
            "b": dep["a"],
            "c": dep["b"],
            "d": dep["c"],
            "e": dep["d"] | t1dep,
            "f": dep["e"],
            "g": dep["f"],
            "h": dep["g"],
        }
        rows.append({"round": r + 1, **{lane: int(initial_word_index in dep_next[lane]) for lane in "abcdefgh"}})
        dep = dep_next
    return pd.DataFrame(rows)

def state_angle_from_trace(trace_df: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for _, row in trace_df.iterrows():
        carrier = float(row["carry_T1_hw"])
        signal = float(row["carry_T2_hw"])
        theta = math.atan2(signal, max(carrier, 1e-12))
        deviation = abs(theta - MARK1_H)
        proxy_gip = row["round"] * MARK1_H + PHI_CONJ * deviation
        rows.append({
            "round": int(row["round"]),
            "carrier_hw": carrier,
            "signal_hw": signal,
            "theta": theta,
            "abs_deviation_from_H": deviation,
            "proxy_gip": proxy_gip,
        })
    return pd.DataFrame(rows)

def operator_tensor():
    rows = []
    for i, src in enumerate(PRIMITIVES, start=1):
        for j, dst in enumerate(PRIMITIVES, start=1):
            rows.append({"row": i, "col": j, "source": src, "target": dst, "token": f"{src}→{dst}"})
    return pd.DataFrame(rows)

print(f"Configured. EXPORT_ARTIFACTS={EXPORT_ARTIFACTS}")
print(f"Mark 1 attractor H = pi/9 = {MARK1_H:.12f}")

```

Configured. EXPORT_ARTIFACTS=False

Mark 1 attractor $H = \pi/9 = 0.349065850399$

0. Reference scaffolds

```
In [3]: ontological_df = pd.DataFrame([
    {"classical": "Nouns / objects", "framework": "Verbs / operations"},
    {"classical": "Linear stack", "framework": "Recursive spiral"},
    {"classical": "Passive space", "framework": "Active computational medium"},
    {"classical": "Matter as substance", "framework": "Matter as frozen verb"},
    {"classical": "Laws as externals", "framework": "Laws as execution traces"},
])

primitive_df = pd.DataFrame({
    "index": np.arange(1, 10),
    "primitive": PRIMITIVES,
    "role": [
        "Directional emission into phase space",
        "Mirrored return across a boundary",
        "Core integration of temporal states",
        "Thermodynamic or scaling dissipation",
        "Constraint collapse / admissibility gate",
        "Outward derivation of a localized noun",
        "Localization at a resonance node",
        "Phase alignment across recursive layers",
        "Final lawful-fit confirmation",
    ]
})

levels_df = pd.DataFrame([
    {"level": 0, "name": "Ground witness", "paper_role": "T2_0 ground fold invariant"},
    {"level": 1, "name": "Word support", "paper_role": "Lane-level support transport"},
    {"level": 2, "name": "Bit support", "paper_role": "Carry-closure support geometry"},
    {"level": 3, "name": "Exact carry automaton", "paper_role": "Constant-generated bit geometry"},
    {"level": 4, "name": "Differential reach", "paper_role": "Exact sensitivity radius"},
    {"level": 5, "name": "Age-weight law", "paper_role": "Tail equalization across rounds"},
    {"level": 6, "name": "Residual smoothing", "paper_role": "Stable density band"},
    {"level": 7, "name": "Removal core", "paper_role": "Invariant subtraction kernel"},
])
```

```
display(ontological_df)
display(primitive_df)
display(levels_df)
```

	classical	framework
0	Nouns / objects	Verbs / operations
1	Linear stack	Recursive spiral
2	Passive space	Active computational medium
3	Matter as substance	Matter as frozen verb
4	Laws as externals	Laws as execution traces

	index	primitive	role
0	1	PROJECT	Directional emission into phase space
1	2	REFLECT	Mirrored return across a boundary
2	3	FOLD	Core integration of temporal states
3	4	LEAK	Thermodynamic or scaling dissipation
4	5	GATE	Constraint collapse / admissibility gate
5	6	BRANCH	Outward derivation of a localized noun
6	7	PIN	Localization at a resonance node
7	8	SYNC	Phase alignment across recursive layers
8	9	VERIFY	Final lawful-fit confirmation

	level	name	paper_role
0	0	Ground witness	T2_0 ground fold invariant
1	1	Word support	Lane-level support transport
2	2	Bit support	Carry-closure support geometry
3	3	Exact carry automaton	Constant-generated bit geometry
4	4	Differential reach	Exact sensitivity radius
5	5	Age-weight law	Tail equalization across rounds
6	6	Residual smoothing	Stable density band
7	7	Removal core	Invariant subtraction kernel

1. The 9-primitive basis and the 81-pair operator tensor

```
In [4]: tensor_df = operator_tensor()
        tensor_grid = tensor_df.pivot(index="source", columns="target", values="row")
        tensor_grid
```

Out[4]: **target** **BRANCH** **FOLD** **GATE** **LEAK** **PIN** **PROJECT** **REFLECT** **SYNC** **VERIFY**

source

BRANCH	6	6	6	6	6	6	6	6	6
FOLD	3	3	3	3	3	3	3	3	3
GATE	5	5	5	5	5	5	5	5	5
LEAK	4	4	4	4	4	4	4	4	4
PIN	7	7	7	7	7	7	7	7	7
PROJECT	1	1	1	1	1	1	1	1	1
REFLECT	2	2	2	2	2	2	2	2	2
SYNC	8	8	8	8	8	8	8	8	8
VERIFY	9	9	9	9	9	9	9	9	9

```
In [5]: fig_tensor = px.imshow(
        tensor_grid,
        aspect="auto",
        labels=dict(color="row index"),
        title="81-element operator tensor scaffold"
    )
fig_tensor.update_layout(height=540)
fig_tensor.show()
```

2. Double-channel die geometry

```
In [6]: trace_nop = compress_trace([0] * 16)
trace_abc = compress_trace(words_from_message(b"abc"))

channel_summary = pd.DataFrame([
    {"channel": "Value channel", "observable": "T1/T2/state words", "meaning": "Explicit round-state compilation"},
    {"channel": "Shape channel", "observable": "carry words / carry counts / residuals", "meaning": "Retained geometr
])
```

```

ground_summary = pd.DataFrame([
    {"quantity": "T2_0", "value_hex": hex(int(trace_nop.loc[0, "T2"])),
    {"quantity": "NOP carry_T2 signature weight", "value": int(trace_nop["carry_T2_count"].sum()),
    {"quantity": "NOP carry_T1 mean hw", "value": float(trace_nop["carry_T1_hw"].mean()),
    {"quantity": "NOP carry_T2 mean hw", "value": float(trace_nop["carry_T2_hw"].mean()),
])

display(channel_summary)
display(ground_summary)

```

	channel	observable	meaning
0	Value channel	T1/T2/state words	Explicit round-state compilation
1	Shape channel	carry words / carry counts / residuals	Retained geometric exhaust

	quantity	value_hex	value
0	T2_0	0x8909ae5	NaN
1	NOP carry_T2 signature weight	NaN	34.000000
2	NOP carry_T1 mean hw	NaN	30.281250
3	NOP carry_T2 mean hw	NaN	16.203125

```

In [7]: fig_die = go.Figure()

nodes = {
    "a,b,c": (0.10, 0.78),
    "e,f,g,h": (0.10, 0.28),
    "K[i], W[i]": (0.10, 0.08),
    "T2": (0.42, 0.78),
    "T1": (0.42, 0.22),
    "a[i+1]": (0.80, 0.78),
    "e[i+1]": (0.80, 0.22),
}

edges = [
    ("a,b,c", "T2"),
    ("e,f,g,h", "T1"),
    ("K[i], W[i]", "T1"),
    ("T2", "a[i+1]"),

```

```

        ("T1", "a[i+1]"),
        ("T1", "e[i+1]"),
    ]
    for src, dst in edges:
        x0, y0 = nodes[src]
        x1, y1 = nodes[dst]
        fig_die.add_annotation(
            x=x1, y=y1, ax=x0, ay=y0,
            xref="x", yref="y", axref="x", ayref="y",
            showarrow=True, arrowhead=3, arrowsize=1, arrowwidth=1.5
        )
    for name, (x, y) in nodes.items():
        fig_die.add_trace(go.Scatter(
            x=[x], y=[y], mode="markers+text", text=[name], textposition="middle center",
            marker=dict(size=34)
        ))
    fig_die.update_xaxes(visible=False, range=[0, 1])
    fig_die.update_yaxes(visible=False, range=[0, 1], scaleanchor="x")
    fig_die.update_layout(title="SHA-256 die viewed as a dual-channel bridge", showlegend=False, height=500)
    fig_die.show()

```

3. Waist geometry and support transport

```

In [8]: injection_vector = np.array([1, 0, 0, 0, 1, 0, 0, 0], dtype=int)
        waist_df = pd.DataFrame([
            {"invariant": "injection vector b", "value": str(injection_vector.tolist())},
            {"invariant": "non-zero entries", "value": int(injection_vector.sum())},
            {"invariant": "topological waist", "value": int(injection_vector.sum())},
            {"invariant": "Mark 1 attractor H", "value": MARK1_H},
        ])
        waist_df

```

Out[8]:

	invariant	value
0	injection vector b	[1, 0, 0, 0, 1, 0, 0, 0]
1	non-zero entries	2
2	topological waist	2
3	Mark 1 attractor H	0.349066

```
In [9]: support_df = word_support_history(initial_word_index=0, rounds=8)
fig_support = px.imshow(
    support_df.set_index("round").T,
    aspect="auto",
    labels=dict(color="dependency active"),
    title="Word-level support propagation from a W0 injection"
)
fig_support.update_layout(height=420)
fig_support.show()

support_df
```

Out[9]:

	round	a	b	c	d	e	f	g	h
0	1	1	0	0	0	1	0	0	0
1	2	1	1	0	0	1	1	0	0
2	3	1	1	1	0	1	1	1	0
3	4	1	1	1	1	1	1	1	1
4	5	1	1	1	1	1	1	1	1
5	6	1	1	1	1	1	1	1	1
6	7	1	1	1	1	1	1	1	1
7	8	1	1	1	1	1	1	1	1

4. A-Mark9 executable checks

```

In [10]: prime_list = primes(64)
prime_audit = pd.DataFrame({
    "index": np.arange(64),
    "prime": prime_list,
    "K_hex": [hex(k) for k in K64],
    "expected_hex": [hex(cube_root_fraction_word(p)) for p in prime_list],
})
prime_audit["match"] = prime_audit["K_hex"] == prime_audit["expected_hex"]

nop_sig = "".join(str(int(v)) for v in trace_nop["carry_T2_count"].tolist())

check_rows = []
violations = 0
for msg_idx in range(200):
    tr = compress_trace(words_from_message(random_one_block_message()))
    residuals = tr["sziklai_residual"].astype(int)
    violations += int((residuals != 0).sum())

amark9_df = pd.DataFrame([
    {"check": "Ground witness T2_0", "result": hex(int(trace_nop.loc[0, "T2"])),
    {"check": "NOP T2 carry signature length", "result": len(nop_sig)},
    {"check": "NOP T2 carry signature", "result": nop_sig},
    {"check": "Prime-root rail exact matches", "result": int(prime_audit["match"].sum())},
    {"check": "Prime-root rail total", "result": len(prime_audit)},
    {"check": "Sziklai residual violations across 200 messages", "result": violations},
])

display(amark9_df)
display(prime_audit.head(12))

```

	check	result
0	Ground witness T2_0	0x8909ae5
1	NOP T2 carry signature length	64
2	NOP T2 carry signature	1101111000011010010101000101011010001101011000...
3	Prime-root rail exact matches	64
4	Prime-root rail total	64
5	Sziklai residual violations across 200 messages	0

	index	prime	K_hex	expected_hex	match
0	0	2	0x428a2f98	0x428a2f98	True
1	1	3	0x71374491	0x71374491	True
2	2	5	0xb5c0fbcf	0xb5c0fbcf	True
3	3	7	0xe9b5dba5	0xe9b5dba5	True
4	4	11	0x3956c25b	0x3956c25b	True
5	5	13	0x59f111f1	0x59f111f1	True
6	6	17	0x923f82a4	0x923f82a4	True
7	7	19	0xab1c5ed5	0xab1c5ed5	True
8	8	23	0xd807aa98	0xd807aa98	True
9	9	29	0x12835b01	0x12835b01	True
10	10	31	0x243185be	0x243185be	True
11	11	37	0x550c7dc3	0x550c7dc3	True

```
In [11]: fig_carry = go.Figure()
fig_carry.add_trace(go.Scatter(x=trace_nop["round"], y=trace_nop["carry_T1_hw"], mode="lines+markers", name="NOP carry_T1_hw"))
fig_carry.add_trace(go.Scatter(x=trace_nop["round"], y=trace_nop["carry_T2_hw"], mode="lines+markers", name="NOP carry_T2_hw"))
fig_carry.add_trace(go.Scatter(x=trace_abc["round"], y=trace_abc["carry_T1_hw"], mode="lines", name="abc carry_T1_hw"))
```

```
fig_carry.add_trace(go.Scatter(x=trace_abc["round"], y=trace_abc["carry_T2_hw"], mode="lines", name="abc carry_T2 hw'
fig_carry.update_layout(
    title="Carry geometry across the NOP backbone and a single-block probe",
    xaxis_title="round",
    yaxis_title="Hamming weight",
    height=460
)
fig_carry.show()
```

5. Sarrus isomorphism crosswalk

```
In [12]: sarrus_df = pd.DataFrame([
    {"domain": "Mechanical engineering", "substrate": "6R linkage", "inward": "Joint articulation", "outward": "Phase",
    {"domain": "Digital cryptography", "substrate": "SHA-256 die", "inward": "Maj", "outward": "Ch", "anchor": "Prime",
    {"domain": "Biological kinematics", "substrate": "Proteins / DNA", "inward": "Alpha-helix bias", "outward": "Beta",
])
sarrus_df
```

```
Out[12]:
```

	domain	substrate	inward	outward	anchor
0	Mechanical engineering	6R linkage	Joint articulation	Phase-shift extension	Guideway / symmetry
1	Digital cryptography	SHA-256 die	Maj	Ch	Prime K-constants
2	Biological kinematics	Proteins / DNA	Alpha-helix bias	Beta-sheet extension	Hydrophobic residues

```
In [13]: labels = [
    "Mechanical", "Cryptographic", "Biological",
    "Inward fold", "Outward extension", "Constraint anchor"
]
source = [0, 1, 2, 0, 1, 2, 0, 1, 2]
target = [3, 3, 3, 4, 4, 4, 5, 5, 5]
value = [1]*9

fig_sarrus = go.Figure(data=[go.Sankey(
    node=dict(label=labels, pad=18, thickness=18),
    link=dict(source=source, target=target, value=value)
)])
```

```
fig_sarrus.update_layout(title="Sarrus isomorphism: three substrates, one grammar", height=520)
fig_sarrus.show()
```

6. BBP random access and the universal ROM picture

```
In [14]: test_positions = [0, 8, 16, 24, 32, 64, 128]
bbp_df = pd.DataFrame([
    {"hex_digit_offset": d, "word_hex": f"0x{bbp_word32(d):08x}", "word_uint32": bbp_word32(d)}
    for d in test_positions
])
bbp_df
```

```
Out[14]:
```

	hex_digit_offset	word_hex	word_uint32
0	0	0x243f6a88	608135816
1	8	0x85a308d3	2242054355
2	16	0x13198a2e	320440878
3	24	0x03707344	57701188
4	32	0xa4093822	2752067618
5	64	0x452821e6	1160258022
6	128	0x9216d5d9	2450970073

```
In [15]: fig_bbp = go.Figure()
fig_bbp.add_trace(go.Bar(
    x=[str(d) for d in bbp_df["hex_digit_offset"]],
    y=bbp_df["word_uint32"],
    text=bbp_df["word_hex"],
    textposition="outside",
    name="BBP 32-bit word"
))
fig_bbp.update_layout(
    title="Random-access 32-bit words from the BBP pointer engine",
    xaxis_title="hex-digit offset",
    yaxis_title="uint32 value",
)
```

```

        height=460
    )
    fig_bbp.show()

```

7. Mark 1 attractor and an H-band phase observable

```

In [16]: mark1_df = pd.DataFrame(
    {
        "quantity": ["pi/9", "0.35", "7/20", "abs(pi/9 - 0.35)", "abs(pi/9 - 7/20)"],
        "value": [
            MARK1_H,
            0.35,
            7/20,
            abs(MARK1_H - 0.35),
            abs(MARK1_H - 7/20),
        ],
    }
)
mark1_df

```

```

Out[16]:

```

	quantity	value
0	pi/9	0.349066
1	0.35	0.350000
2	7/20	0.350000
3	abs(pi/9 - 0.35)	0.000934
4	abs(pi/9 - 7/20)	0.000934

```

In [17]: theta = np.linspace(0, 2*np.pi, 600)
fig_mark1 = go.Figure()
fig_mark1.add_trace(go.Scatter(x=np.cos(theta), y=np.sin(theta), mode="lines", name="unit circle"))
fig_mark1.add_trace(go.Scatter(
    x=[0, math.cos(MARK1_H)],
    y=[0, math.sin(MARK1_H)],
    mode="lines+markers",
    name="H = pi/9"
))

```

```

))
fig_mark1.update_layout(
    title="Mark 1 attractor geometry on the unit circle",
    xaxis_title="x",
    yaxis_title="y",
    yaxis_scaleanchor="x",
    height=560
)
fig_mark1.show()

phase_df = state_angle_from_trace(trace_nop)
phase_df.head()

```

Out[17]:

	round	carrier_hw	signal_hw	theta	abs_deviation_from_H	proxy_gip
0	0	27.0	23.0	0.705568	0.356502	0.220331
1	1	31.0	18.0	0.526066	0.177001	0.458458
2	2	32.0	18.0	0.512389	0.163324	0.799071
3	3	32.0	20.0	0.558599	0.209533	1.176696
4	4	30.0	20.0	0.588003	0.238937	1.543934

```

In [18]: fig_phase = go.Figure()
fig_phase.add_trace(go.Scatter(
    x=phase_df["round"],
    y=phase_df["theta"],
    mode="lines+markers",
    name="theta(round)"
))
fig_phase.add_hline(y=MARK1_H, line_dash="dash", annotation_text="H = pi/9")
fig_phase.update_layout(
    title="Round-wise phase observable against the Mark 1 attractor",
    xaxis_title="round",
    yaxis_title="angle (radians)",
    height=440
)
fig_phase.show()

phase_summary = pd.DataFrame([

```

```

    "mean_theta": float(phase_df["theta"].mean()),
    "mean_abs_deviation_from_H": float(phase_df["abs_deviation_from_H"].mean()),
    "mean_proxy_gip_gap": float(np.diff(np.sort(phase_df["proxy_gip"])).mean()),
  })
phase_summary

```

```

Out[18]:
   mean_theta  mean_abs_deviation_from_H  mean_proxy_gip_gap
0      0.486966                0.15263                0.346116

```

8. Optional curated export

```

In [19]: if EXPORT_ARTIFACTS:
    primitive_df.to_csv(OUTPUT_DIR / "primitive_basis.csv", index=False)
    levels_df.to_csv(OUTPUT_DIR / "amark9_levels.csv", index=False)
    support_df.to_csv(OUTPUT_DIR / "word_support_history_W0.csv", index=False)
    prime_audit.to_csv(OUTPUT_DIR / "prime_root_audit.csv", index=False)
    bbbp_df.to_csv(OUTPUT_DIR / "bbp_word_reads.csv", index=False)
    phase_df.to_csv(OUTPUT_DIR / "phase_observable.csv", index=False)
    print(f"Exports written to: {OUTPUT_DIR.resolve()}")
else:
    print("EXPORT_ARTIFACTS is False – no files written.")

```

EXPORT_ARTIFACTS is False – no files written.

9. Next checks

This notebook captures the paper's strongest executable spine. The obvious next extensions are:

1. implement a paper-exact AHRC-old / AHRC-new module if the missing equation renderings are restored,
2. add a removal-core lab for late-seam falsification tests,
3. attach RGBA Math-Light closure experiments to a fully specified field metric,
4. compare multiple probe classes against the same NOP basin.